Technical Challenge: Dynamic Locker Layout

Auto-detection of the locker's physical door layout using existing hardware

Context

- Physical lockers are a matrix of compartments (columns), each column has rows **(A..Z × 1..N).**
- Each compartment has an electronic lock controlled by a *lock controller board* connected on an RS-485 **Modbus RTU** bus.
- A lock controller has a **unique device address** (DIP) and drives N locks (e.g., 48, 24, 16).
- When physical layout changes in the field, we must reconstruct the physical layout from the electronics and electrical view point — **manual mapping for every variant**.
- Replacements in field **(board swap)** should be detected and the previously stored mapping for that board id used (or migrated).

Requirements

A service that talks Modbus/RS485, auto-detects boards on the bus, reads board metadata and status, and reconstructs a JSON layout describing columns/rows/compartments.

- **Bus scan – Search for all lock boards on the RS485 bus**
- **Read board info – For each board, get all available info such as version and door layout.**
- **Aggregate and infer layout – Using each module's door layout, build the overall locker layout.**
- **Persist mapping per board – If a lock control board is replaced, the locker sends the stored layout to the new board.**

The goal is to remove manual mapping and allow automatic reconstruction of the physical layout purely from RS-485 discovery.

```
RS-485 bus
  ├── Board 0  (48 locks)
  ├── Board 1  (48 locks)
  └── Board 2  (24 locks)
          |
          ▼
   [ your service ]
          |
          ▼
  {
    "columns": [
      { "rows": [ { "compartments": [{
"label": "A1", ... }] } ] }
    ]
  }
          |
   open("A1") → unlock board 0, lock 0
```

Lock System Flow

Solution Architecture

1/ 2 Delivery

Architecture and Components

Before writing any code, produce a short document (can be a diagram + bullet points) that answers:

1. What are the components and what does each one own?
2. What is the data contract between them?
   - What does the Board Scanner return?
   - What does the Layout Builder consume and produce?
   - What does the Mapping Store persist and in what format?
3. How does `open("A1")` flow through the system?
4. What happens if a board that was previously seen disappears from the bus?
5. What happens if a new board appears that was not in the stored mapping?

> There is no single correct answer.
> The proposal is evaluated on clarity of boundaries and completeness of reasoning.

Solution Architecture

1/ 2 Delivery

Architecture and Components – Board Scanner

Scans board addresses `0` to `15`, identifies which respond, and reads their type.

Expected output (your own data structure — propose it):

```
BoardInfo
  address:   int           # 0-based
  model:     str           # "NCU48L", "NCU24L", "NCU12L"
  capacity:  int           # number of locks: 48 / 24 / 12
  status:    dict[int, bool]  # lock_id → is_open at scan time
```

Requirements:

- A board that does not respond to Get Status is considered absent
- If Read Board Type fails, default capacity to **48** and log a warning
- Scanner must be callable repeatedly — each call returns current bus state

Solution Architecture

1/ 2 Delivery

Architecture and Components - Mapping Index Inferences

```
Row label assignment:
  lock index 0 on the first board of a column → row "A"
  lock index 1                                → row "B"
  ... up to "Z", then "AA", "AB", ...

Column number assignment:
  first  column → "1"
  second column → "2"
  ...

Compartment label:
  row_label + column_number  →  "A1", "B1", "A2", ...
```


```
{
  "columns": [
    {
      "rows": [
        {
          "compartments": [
            {
              "label": "A1",
              "boardId": "1",
              "lockId": "1",
              "openDirection": "right",
              "size": "M"
            }
          ]
        },
        {
          "compartments": [
            {
              "label": "D1",
              "boardId": "1",
              "lockId": "4",
              "openDirection": "right",
              "size": "XS"
            },
            {
              "label": "E1",
              "boardId": "1",
              "lockId": "5",
              "openDirection": "left",
              "size": "XS"
            }
          ]
        }
      ]
    }
  ]
}
```


JSON Door mapping example

Solution Architecture

1/ 2 Delivery

Architecture and Components – Door Service

Exposes a single operation:

```python
def open(label: str) -> OpenResult: ...
```

`OpenResult` is an enum you define. At minimum: `OK`, `NOT_FOUND`, `TIMEOUT`.

- If label is not in the mapping: return `NOT_FOUND` immediately
- If board is marked offline: return a result that makes the offline state visible
- No retry logic required

Flow:

```
open("A1")
    │
    ├─ Look up "A1" in stored mapping
    │   → boardId=0, lockId=0
    │
    ├─ Send Unlock frame to board 0, lock 0
    │
    ├─ Poll board 0 status
    │
    └─ Return OK if lock 0 is now open, TIMEOUT otherwise
```

Open Door Flow

Solution Architecture

1/ 2 Delivery

In slides or markdown docs, explain your solution architecture and core components using a fluxogram and explaining responsibilities of each main component and assumptions that were made to make this project. Please consider in your project the following points in your final solution.

Developer Notes

Device Addresses

Create a blacklist for bus addresses used by non-lock controllers.

Modbus CRC16

Respect Modbus CRC16 (low byte first).

Retries & Backoff

No need to Implement retries and exponential backoff for flaky boards.

Layout Reconstructor

- Uses boards mapping + simple heuristics to compose columns and rows.
- Maps board lock index → compartment label (A1..), computes openDirection, size

Persistence & Migration

- Store mapping per-board-id (serial + model) locally.
- On new board detection: if previously known board-id found, reapply its stored map; else apply reconstruction heuristics.

Tech Assumptions

- Assume 1 Board = 1 Column.
- Always start row labels at "A" for each new board and increment numerically (A1, A2...).

Deliverables & Test Plan

2/ 2 Delivery

Deliverables (ZIP contents)

`src/` — application source
`setup_install.sh` + `requirements.txt`
`README.md` — install & run instructions
`doc/` — PDF and/or markdown exported

Evaluation Notes / Bonus Points

- Clean code, CI, and docker/sandboxed demo gets bonus.
- Clear demonstration of mapping algorithm (diagram + complexity).

Test Cases

Simple single-board detection — 1 board (48 locks) returns correct locks count and mapping → PASS.
Mixed boards — bus contains 48+24 boards; layout groups them into columns correctly → PASS.
Board replacement — replace a board ID with same board type; stored mapping is reapplied → PASS.

Acceptance Criteria (Score-Based Rubric)

Area	Weight	Criteria
Functional correctness	40%	Autodetect & reconstruct layout; handles replacements & mixed types
Architecture Planning	40%	Structure and scalability view
Code clarity and Docs	20%	Clear README, demo steps, install script

Project Support

bridge-server-client

A ready-to-use client. You send bytes, you get bytes back.
You do not implement serial port handling.

- **Initial Setup:** CPU app (service) ⇄ Serial Bridge / Socat ⇄ physical RS-485 emulator — Abstracted serial bridge / client provided by Bloq.it can be used as a pyserial.Serial or a socat-backed PTY, allowing to create a local virtual serial port; Git clone this repo: https://github.com/brunofbloq/serial-bridge-client — will require a machine with access to internet.

Important Notes:

- **Server URL:** https://kerong-modbus-emulator.vercel.app/
- **Authentication:** User and password credentials are provided at `<server-url>/user/login/`
- **Support:** If you encounter issues running the executable or have questions, please contact bruno.figueiredo@bloq.it.

Appendix 1

1. Communication Parameters (RS485)

Baudrate: 19200 bps
Data: 8 bits
Parity: None
Stop bits: 1
Endian: Big-Endian (high byte first), except CRC

---

2. Register Map

2.1 Identification and Monitoring (Read — FC 0x03)

Address (Hex)	Function	Description	Example (Hex)
0x000F	Board ID	High byte: model (e.g., 0xE0). Low byte: max capacity (e.g., 0x30 = 48).	`01 03 00 0F 00 01 B4 09`
0x00F5	Firmware Version	Returns SW (Hi) and HW (Lo).	`01 03 00 F5 00 01 94 1B`
0x00F7	Voltage (mV)	Vbat reading in mV (e.g., 12000).	`01 03 00 F7 00 01 35 D7`
0x00F2	Baud rate	0 = 9600, 1 = 19200.	`01 03 00 F2 00 01 25 F9`
0x0000	Lock Status	Bits 0..n represent locks (0 = open, 1 = closed).	`01 03 00 00 00 01 84 0A`
0x0090–0093	IR Counter	Count per IR port (1–4).	`01 03 00 90 00 01 84 27`

Appendix 2

2.2 Action Control (Write — FC 0x05 / 0x10)

Address (Hex)	Function	Description	Example (Hex)
0x0000–002F	Open (bit)	FC 0x05. Bit address = port no. - 1. Write `FF 00` to open.	`01 05 00 00 FF 00 8C 3A` (Port 1)
0x0000–0002	Open (reg)	FC 0x10. Activates multiple ports sequentially.	`01 10 00 00 00 01 02 00 00 A6 50` (1–16)
0x00F1	Initialize	FC 0x10. Factory reset.	`01 10 00 F1 00 01 02 00 00 B3 71`
0x00F3	OTA Mode	FC 0x10. Enters update. Password `0x35363738`.	`01 10 00 F3 00 02 04 35 36 37 38 44 DE`
0x0090	Clear IR	FC 0x06. Resets IR counter.	`01 06 00 90 00 00 89 E7`

2.3 Configuration (Read/Write — FC 0x03 / 0x10)

Address (Hex)	Function	Description	Example (Hex)
0x00F0	Opening Time	Unit: 10 ms (e.g., `0x64` = 1000 ms).	`01 10 00 F0 00 01 02 00 64 B3 4B`
0x00F8	LED Time	Unit: 1 s (default 5 s).	`01 10 00 F8 00 01 02 00 64 B2 03`
0x0070–0079	User Data	Free registers for app use.	`01 06 00 70 12 34 85 66`

Appendix 3

3. Hardware Identification (High Byte of 0x000F)

Reading `0x000F` — high byte maps to the model (source: manual p.237):

Code (Hex)	Model
0xE0	48
0xE1	48 Infrared
0xE2	48 Low Power - Standard
0xE4	48 Low Power
0xA0	24
0x80	16
0x40	12 / 8

4. Important Notes

Addressing: Slave Address = DIP switch (hardware code) + 1.

Broadcast: Address `0x00` sends to all boards (no reply).

CRC: Always terminate with Modbus CRC-16 (Low byte first, then High).

Appendix 4

4. Cyclic Redundancy Check (CRC)

CRC code example

```
# ---------------------------------------------------------------------------
# CRC helpers
# ---------------------------------------------------------------------------

def crc16_modbus(data: bytes) -> int:
    crc = 0xFFFF
    for b in data:
        crc ^= b
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0xA001
            else:
                crc >>= 1
    return crc & 0xFFFF


def append_crc(frame: bytes) -> bytes:
    c = crc16_modbus(frame)
    return frame + bytes([c & 0xFF, (c >> 8) & 0xFF])


def check_crc(frame: bytes) -> bool:
    if len(frame) < 3:
        return False
    data, crc = frame[:-2], frame[-2:]
    c = crc16_modbus(data)
    return (crc[0] == (c & 0xFF)) and (crc[1] == ((c >> 8) & 0xFF))
```

Supporting references:

- Serial Port Communication
- RS485 Protocol
- Modbus Serial Protocol
- Cyclic Redundancy Check (CRC)